FOOD ORDER MANAGEMENT SYSTEM

Implementation using a Doubly Linked List in C++

1. Topic

Food Order Management System using a Doubly Linked List - a menu-driven C++ program to place, display, search and cancel food orders and to calculate the total bill.

2. Introduction

Restaurants, canteens and food counters receive orders continuously. Orders are added in the order they arrive, and some of them are cancelled before they are served. The number of orders is not known in advance, so a fixed-size array wastes memory and makes deletion from the middle slow. A **doubly linked list** solves this: each order is a node created dynamically, every node stores the address of both its previous and next node, and the list can be read from the first order to the last or from the last order back to the first. Deleting any order only needs a few pointer changes.

3. Problem Statement

Design and implement a **Food Order Management System** in C++ using a **Doubly Linked List** that allows a user to place, view, search and cancel food orders from a predefined menu, and to calculate the total bill.

Details

- A fixed menu of 6 items (Pizza, Burger, Sandwich, French Fries, Dosa, Cold Coffee) with prices is stored in arrays.
- Each order is stored as a node in a doubly linked list with: Order ID (auto-generated), food item, quantity, amount, and *prev* / *next* pointers.

No.	Food Item	Price (Rs.)
1	Pizza	150
2	Burger	80
3	Sandwich	60
4	French Fries	70
5	Dosa	50
6	Cold Coffee	90

Table 1: Predefined food menu

Operations to implement

1. **Add order:** insert a new node at the end of the list (with validation of food number and quantity).
2. **Display all orders:** traverse forward from head and show the total bill.
3. **Display in reverse:** traverse backward from tail.
4. **Search order:** find an order by Order ID.
5. **Cancel order:** delete a node by Order ID, updating the prev/next links, head and tail.
6. **Calculate total bill:** sum the amounts of all orders.
7. **Display menu:** show the food list with prices.
8. **Exit:** free all dynamically allocated memory.

4. Objectives

Main objective: To apply the concepts of dynamic memory allocation, structures, and doubly linked list operations (insertion, deletion, forward and backward traversal, searching) in a menu-driven real-life application.

Specific objectives:

- To use **structures** and **pointers** to build a node holding data and two links.
- To use **dynamic memory allocation** (*new* / *delete*) to create and remove nodes at run time.
- To insert at the end of the list in O(1) time using the *tail* pointer.
- To delete a node from the head, middle or tail by correctly updating the links, head and tail.
- To implement forward and backward traversal and linear search.
- To validate user input so invalid food numbers and quantities are rejected.
- To prevent **memory leaks** by releasing every node before the program exits.
- To develop a modular, menu-driven program that solves a practical problem.

5. Data Structure Used

Each order is one node of a doubly linked list. Two global pointers, *head* and *tail*, mark the first and last node, and *idCounter* generates unique Order IDs starting from 1.

```
struct Node {
    int   orderID;    // auto-generated
    int   item;       // index into food[] / price[]
    int   quantity;
    int   amount;     // price[item] * quantity
    Node* prev;       // previous order
    Node* next;       // next order
};
```

Structure of the list: head points to the first node whose *prev* is NULL; tail points to the last node whose *next* is NULL. Every node in between links to both neighbours.

6. Algorithm

6.1 Main Program

1. Start. Set head = NULL, tail = NULL, idCounter = 1.
2. Display the main menu (options 1 to 8) and read *choice*.
3. If choice = 1 call placeOrder(); 2 displayOrders(); 3 displayReverse(); 4 searchOrder(); 5 cancelOrder(); 6 calculateBill(); 7 displayMenu().
4. If choice = 8, call clearOrders(), print 'Thank you!' and go to Step 6.
5. Otherwise print 'Invalid choice!' and repeat from Step 2.
6. Stop.

6.2 Add Order (Insert at End)

1. Display the menu and read the food number *choice*.
2. If choice < 1 or choice > 6, print 'Invalid food number!' and return.
3. Read *quantity*. If quantity <= 0, print 'Invalid quantity!' and return.
4. Create a new node: orderID = idCounter++, item = choice - 1, quantity = quantity, amount = price[item] x quantity, prev = tail, next = NULL.
5. If head = NULL, set head = newNode; otherwise set tail->next = newNode.
6. Set tail = newNode.
7. Display the order details (ID, food, price, quantity, amount) and return.

6.3 Display All Orders (Forward Traversal)

1. Set temp = head. If temp = NULL, print 'No orders found!' and return.
2. Set total = 0.
3. While temp != NULL: print its details, add temp->amount to total, set temp = temp->next.
4. Print the total bill and return.

6.4 Display in Reverse (Backward Traversal)

1. Set temp = tail. If temp = NULL, print 'No orders found!' and return.
2. While temp != NULL: print its details and set temp = temp->prev.

6.5 Search Order

1. Read the Order ID *id* and set temp = head.
2. While temp != NULL: if temp->orderID = id, print the order and return; else set temp = temp->next.
3. Print 'Order not found!'.

6.6 Cancel Order (Delete a Node)

1. Read the Order ID *id* and set temp = head.
2. Traverse the list until temp->orderID = id. If temp becomes NULL, print 'Order not found!' and return.
3. If temp->prev != NULL, set temp->prev->next = temp->next; otherwise set head = temp->next.
4. If temp->next != NULL, set temp->next->prev = temp->prev; otherwise set tail = temp->prev.
5. Delete temp, print 'Order cancelled successfully!' and return.

6.7 Calculate Total Bill

1. Set temp = head and total = 0.
2. While temp != NULL: total = total + temp->amount; temp = temp->next.
3. Print total.

6.8 Exit (Free Memory)

1. Set temp = head.
2. While temp != NULL: store nextNode = temp->next, delete temp, set temp = nextNode.
3. Set head = NULL and tail = NULL.

7. Flowcharts

The flowchart below shows the overall working of the menu-driven program.

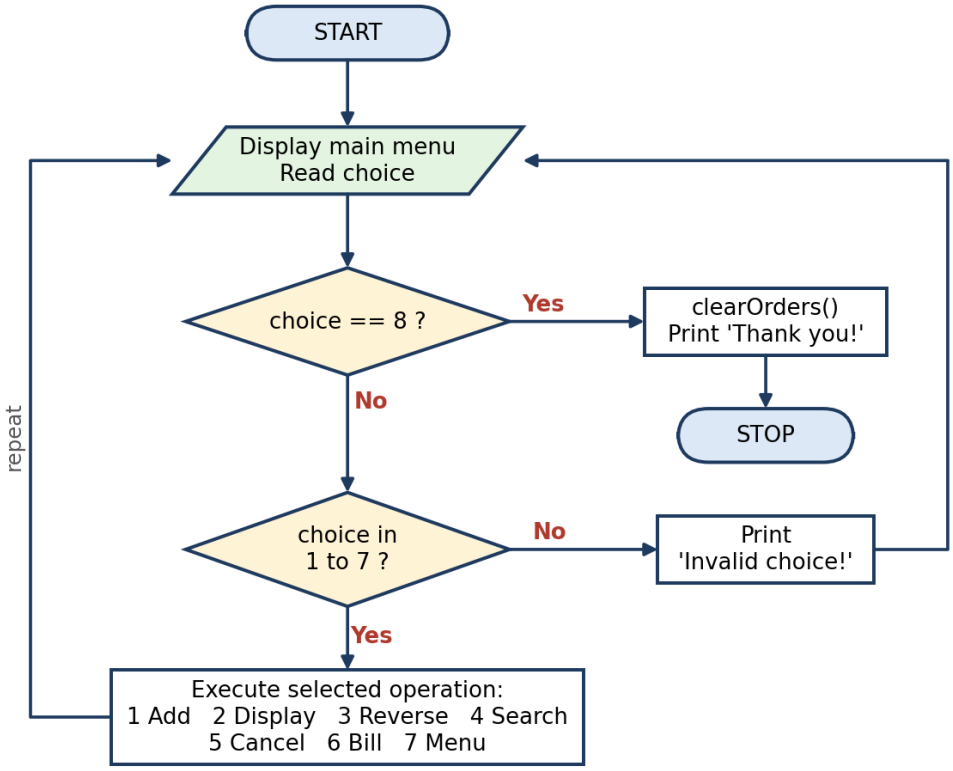


Figure 1: Main program flowchart

Add Order (insertion at the end of the list)

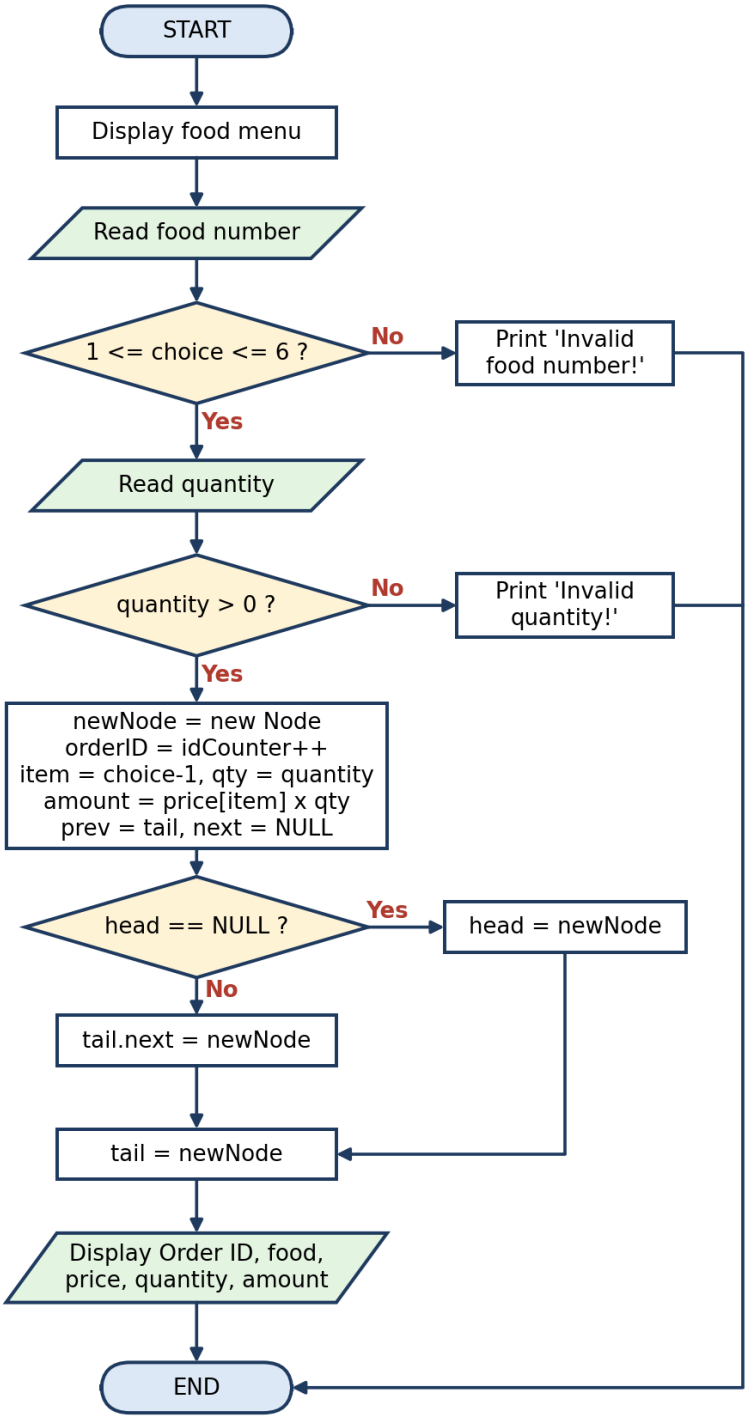


Figure 2: Flowchart of Add Order

Cancel Order (deletion of a node by Order ID)

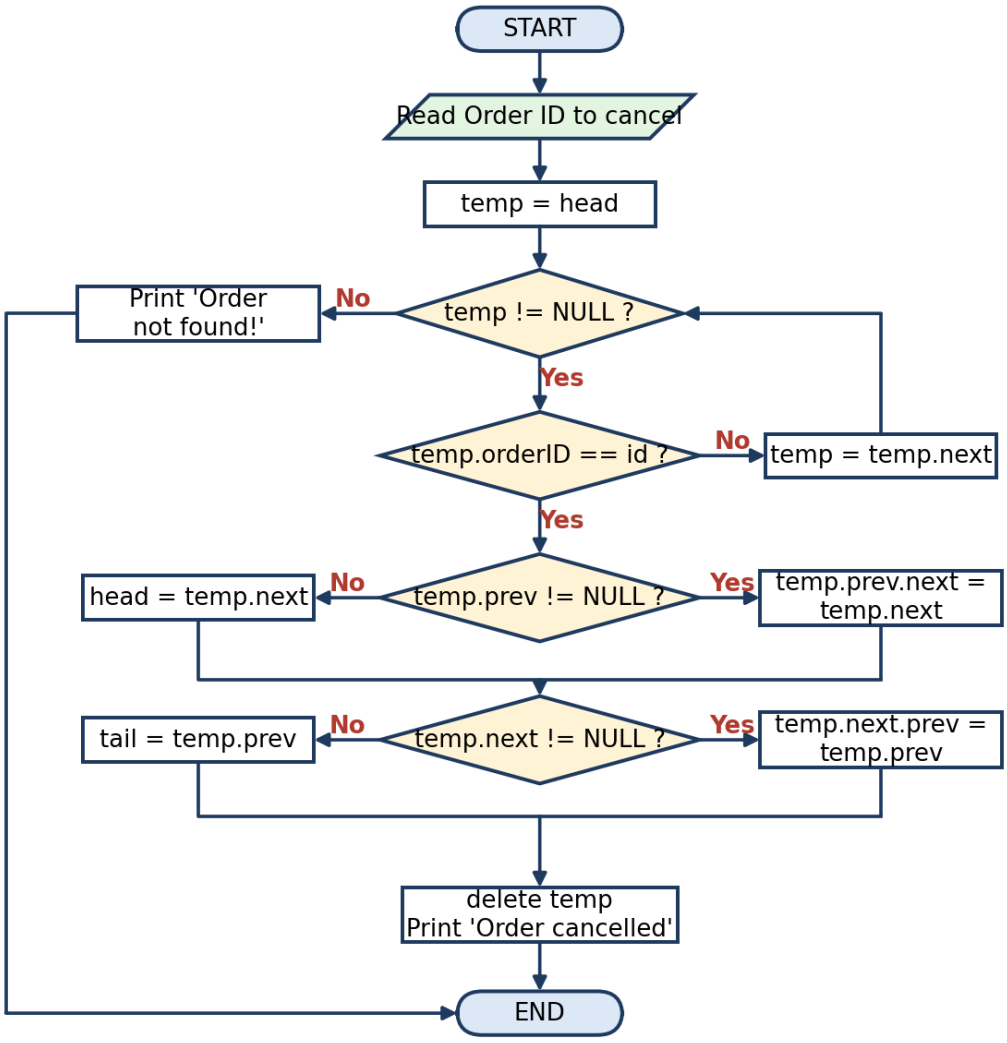


Figure 3: Flowchart of Cancel Order

8. Sample Input and Output

Three orders are placed: 2 Pizza, 1 Burger and 3 Dosa. The orders are then displayed, order 2 is cancelled, and the bill is calculated again.

Step	Input	Result
Add order	Food 1 (Pizza), qty 2	Order ID 1, Amount Rs. 300
Add order	Food 2 (Burger), qty 1	Order ID 2, Amount Rs. 80
Add order	Food 5 (Dosa), qty 3	Order ID 3, Amount Rs. 150
Display all	-	IDs 1, 2, 3 in order; Total Bill Rs. 530
Display reverse	-	IDs 3, 2, 1
Search	Order ID 2	Order found: Burger, Qty 1, Rs. 80
Cancel	Order ID 2	Order cancelled successfully!
Total bill	-	Rs. 450 (orders 1 and 3 remain)
Add order	Food 9, qty 1	Invalid food number!
Add order	Food 3, qty 0	Invalid quantity!
Exit	8	All nodes freed; Thank you!

Table 2: Sample test run

9. Time Complexity of Operations

Operation	Time	Reason
Add order	O(1)	Direct insertion at tail
Display all / reverse	O(n)	Visits each node once
Search order	O(n)	Linear search by Order ID
Cancel order	O(n)	O(n) to find the node, O(1) to unlink it
Calculate bill	O(n)	Sums all amounts
Display menu	O(1)	Menu size is fixed (6 items)
Exit (free memory)	O(n)	Deletes each node once

Table 3: Time complexity (n = number of orders)

The extra memory used is O(n), one node per order, and each node stores two pointers.

10. Advantages of Using a Doubly Linked List

- Memory is allocated only when an order is placed, so there is no fixed limit on orders.
- Insertion at the end is O(1) because the tail pointer is maintained.
- A node can be deleted using its own *prev* and *next* pointers, with no shifting of elements as in an array.
- The list can be traversed in both directions (oldest to newest and newest to oldest).

11. Real-Life Applications

- Restaurant, canteen and college mess billing counters.
- Online food delivery order queues.
- Ticket booking and token systems where entries are added and cancelled.
- Browser history, music playlists and undo/redo features, which use the same two-way linking idea.

12. Limitations and Future Scope

Limitations

- The menu is fixed and stored in arrays, so items and prices cannot be changed at run time.
- Orders are kept only in memory and are lost when the program exits.
- Non-numeric input (for example, a letter instead of a number) is not handled and can disturb the input loop.
- Searching and cancelling take linear time because the list is not indexed.

Future scope

- Save and load orders using file handling.
- Add, edit and remove menu items, and allow several items in one order.
- Add GST, discounts and a printed bill.
- Add order status (preparing, served), customer name or table number, and a graphical interface.

13. Conclusion

The Food Order Management System was successfully designed and implemented in C++ using a doubly linked list. The program places orders from a predefined menu, stores each order as a dynamically allocated node, displays orders in forward and reverse order, searches and cancels orders by Order ID, and calculates the total bill correctly. Deletion works for the first, middle and last node because the *prev*, *next*, head and tail pointers are all updated, and all memory is released when the program exits.

The project shows how pointers, structures and dynamic memory allocation combine to give a flexible data structure that grows and shrinks with the number of orders. It also shows why a doubly linked list suits real-life tasks that need frequent insertion, deletion and two-way traversal. With file handling, an editable menu and a graphical interface, the system could be extended into a complete restaurant billing application.